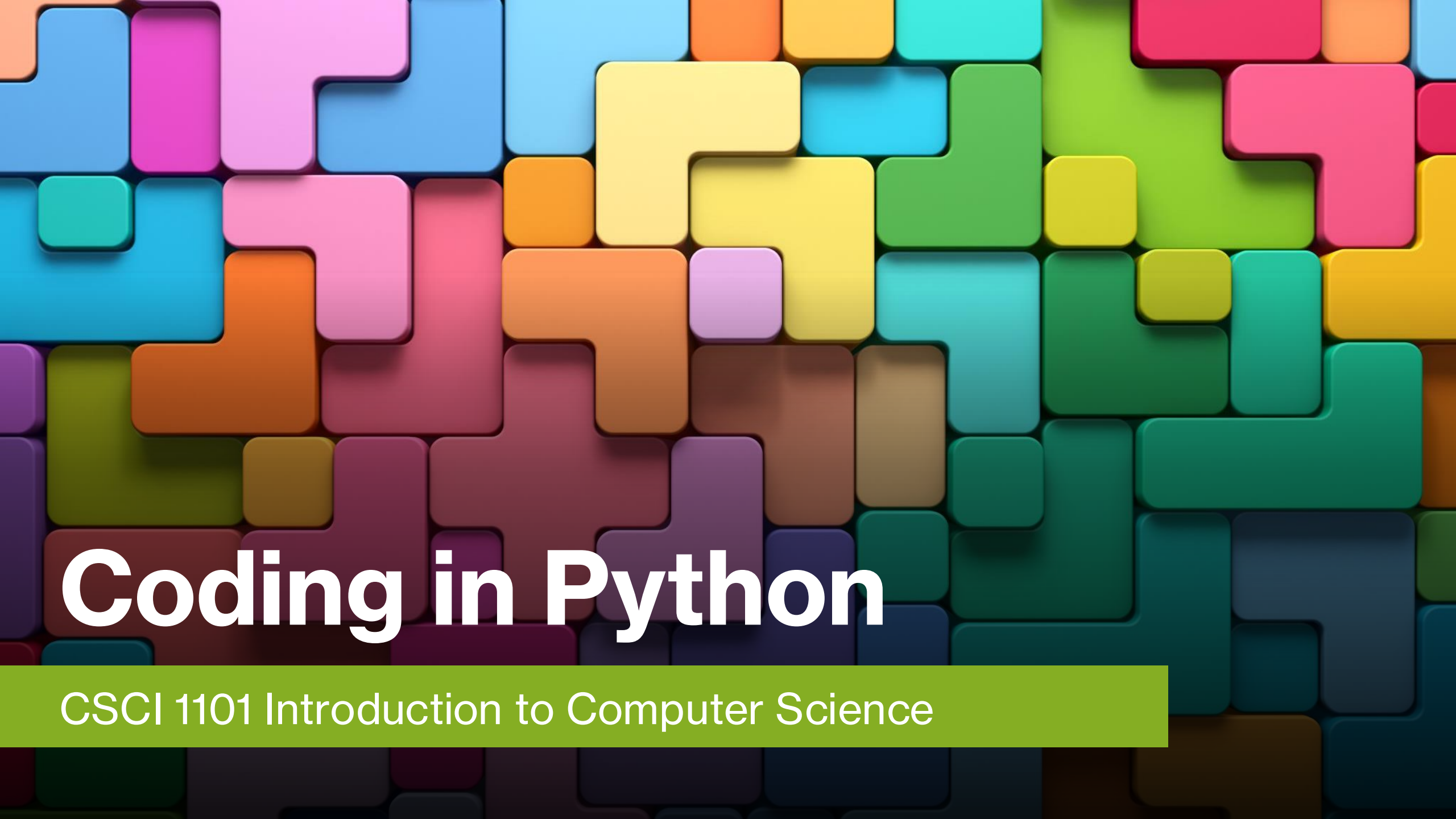




Coding in Python

CSCI 1101 Introduction to Computer Science

Variables

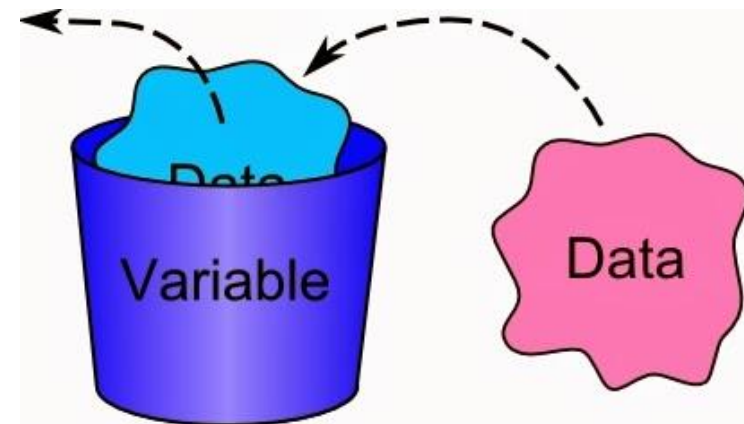
- A variable is a storage location (*identified by a memory address*) paired with an associated symbolic name, which contains some known or unknown quantity of information referred to as a value.

- **Example**

`name` = "John Doe"

`emptyVar` = `name`

`name` = "Jane Doe"



Data Types

- A data type is an attribute of a piece of data that tells a device how the end-user might interact with the data.
- **Basic Data Types**
 - String - "Hello World"/'Hello World'
 - Integer - 2754
 - Float – 10.0247
 - Boolean – True/False

Output Messages

- The **print()** function prints the specified message to the screen, or other standard output device. The message can be a string, or any other object, the object will be converted into a string before written to the screen.

- **Example**

```
print("Hello World") -> Hello World
```

```
var1 = "Hello"
```

```
var2 = 23
```

```
var3 = False
```

```
print(var1, var2, var3) -> Hello 23 False
```

Get User Input

- The **input()** function is used to take user input. By default, it returns the user input in form of a string. It is optional to include a string value to display as an input message.
- Example

```
name = input() <- John Doe
print(name) -> John Doe
name = input("What is your name? ") <- Jane Doe
print(name) -> Jane Doe
```

Changing Data Types

- We cannot perform arithmetic operations with different data types. We can transform string representations of numbers into numbers using `int()` and `float()`, and numbers into strings using `str()`.

- **Example**

```
var1 = "2"
```

```
var2 = 2
```

```
var3 = var1 + var2 -> Error, since var3 = "2" + 2 and we cannot add diff. data types
```

- **Solution**

```
var3 = int(var1) + var2
```

```
print(var3) -> 4
```

```
var3 = var1 + str(var2)
```

```
print(var3) -> 22
```

Decision Making

- Decision Making in programming is similar to decision-making in real life. In programming also face some situations where we want a certain block of code to be executed when some condition is fulfilled.
- **Python Decision Statements**
 - if statement
 - if-else statement
 - nested-if statement
 - if-else-if statement



Decision Making (Cont.)

if Statement

if (*condition*):

Execute code here.

Execute code here.

...

if-else Statement

if (*condition*):

Execute code here.

...

else:

Execute code here.

...

Decision Making (Cont.)

nested-if Statement

```
if ( condition ):  
    # Execute code here.  
  
...  
  
if ( condition ):  
    # Execute code here.  
  
...
```

if-else-if Statement

```
if ( condition ):  
    # Execute code here.  
  
...  
  
elif ( condition ):  
    # Execute code here.  
  
...
```

Decision Making (Cont.)

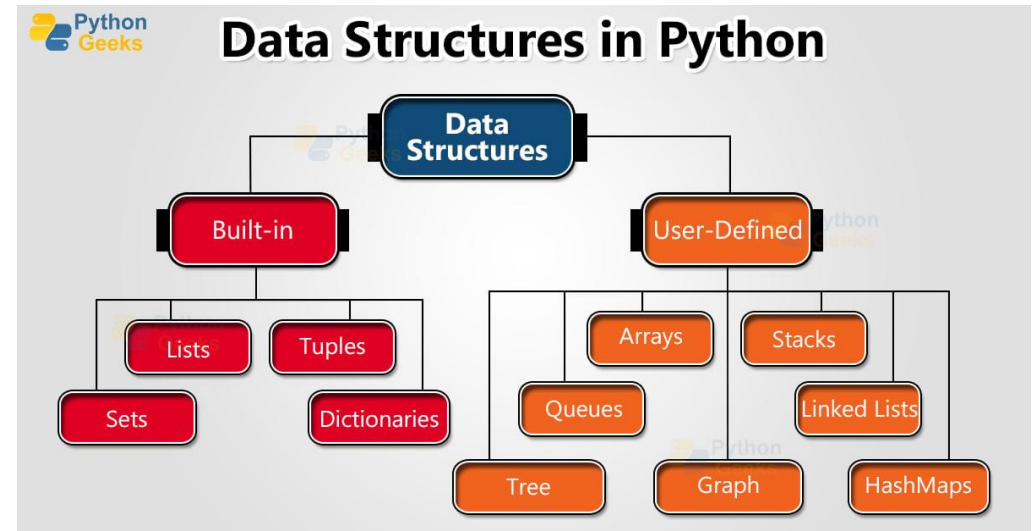
Expression	Definition
<code>a == b</code>	If a is equal to b.
<code>a != b</code>	If a is not equal to b.
<code>a > b</code>	If a is greater than b.
<code>a < b</code>	If a is less than b.
<code>a >= b</code>	If a is greater than, or equal to b.
<code>a <= b</code>	If a is less than, or equal to b.
<code>a and b</code>	If a and b are True.
<code>a or b</code>	If a or b is True.
<code>a</code>	If a is True.
<code>!a</code>	If a is not True.

Logical Conditional Statements

- Logical conditions inside an if or else-if statement must return a boolean value.

Data Structures

- A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.



List

- Lists are used to store multiple items of different data types in a single variable. Items in a list are ordered, changeable, and allow duplicate values. List items are also indexed, the first item has index [0], the second item has index [1] etc.
- **Example**
`myList = [ "Hello", 25, False ]`
`print(myList[0]) -> Hello`
`myList.append("Good Bye")`
`print(myList) -> [ 'Hello', 25, False, 'Good Bye' ]`

Dictionary

- Dictionaries are used to store data values in key:value pairs. A dictionary is ordered*, changeable and does not allow duplicates. In a dictionary key:value pairs can be referred to by using the key name.

- **Example**

```
myDict = { "name": "Bob", "age": 23, "address": "123 S. St" }
```

```
print(myDict["age"]) -> 23
```

```
myDict["height"] = 5.7
```

```
print(myDict) -> {'name': 'bob', 'age': 23, 'address': '123 S. St', 'height': 5.7}
```

Loops

- A Loop is used to execute a group of instructions or a block of code multiple times, without writing it repeatedly. The block of code is executed based on a certain condition. Loops are the control structures of a program.

- **Types of Loops**

- For loops
- While loops

